

Detailed Implementation and Analysis of CNNs on CIFAR10

Zahra Khodagholi

1 Introduction

This document provides a detailed analysis of two Convolutional Neural Network (CNN) architectures designed for image classification on the CIFAR10 dataset. We aim to compare the performance of a baseline CNN model with a modified version that includes additional convolutional layers.

2 CNN Architectures

2.1 Baseline CNN Architecture

The baseline CNN consists of three convolutional layers with increasing filter sizes, batch normalization, max-pooling, dropout, and three fully connected layers. The ReLU activation function is used throughout.

2.2 Modified CNN Architecture

The modified CNN adds two additional convolutional layers and adjusts the fully connected layers accordingly. The intent is to capture more complex features for improved accuracy.

3 Training and Evaluation

Both networks are trained using Stochastic Gradient Descent (SGD) with a learning rate of 0.001 and momentum of 0.9. We utilize cross-entropy loss as our loss function. The training and evaluation process is encapsulated in the function `train_and_evaluate`, which records training and testing loss and accuracy over 30 epochs.

4 Results

4.1 Baseline CNN Results

The baseline CNN achieved the following results, with the losses plotted over epochs shown in Figure 1.

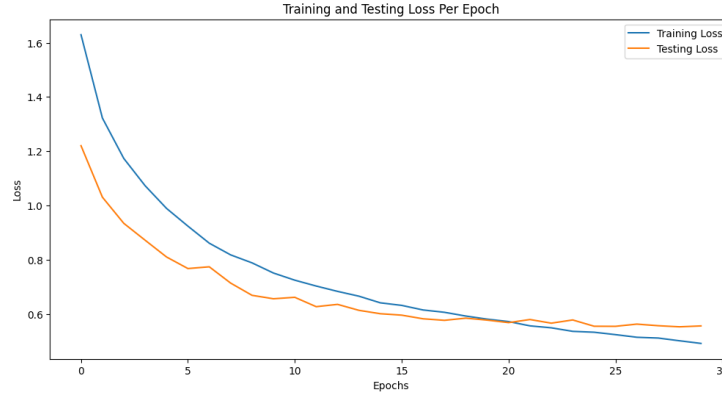


Figure 1: Training and testing loss for the baseline CNN architecture.

This graph shows a smooth and consistent decrease in both training and testing loss as the number of epochs increases. The training loss starts higher and decreases at a faster rate than the testing loss, but they both level off as they approach 30 epochs. There is a small gap between the training and testing loss, which suggests that the model is learning effectively and generalizing well to unseen data.

4.2 Modified CNN Results

The modified CNN, with additional convolutional layers, achieved the following results, with the losses plotted over epochs shown in Figure 2.

The second graph displays a more erratic behavior, particularly in the testing loss. While the training loss decreases smoothly, similar to the first image, the testing loss shows fluctuations and a significant spike around the 25th epoch. This spike indicates a moment during testing where the model performed significantly worse than in other epochs. Such a spike could be the result of overfitting, where the model learns the training data so well that it performs poorly on the test data. It could also be a sign of an unstable training process, perhaps due to a learning rate that's too high, data that isn't properly shuffled, or other issues.



Figure 2: Training and testing loss for the modified CNN architecture.

5 Key Differences

- The first image represents a model that is learning steadily and generalizing well.
- The second image shows a model that might be overfitting and is definitely less stable in its learning process.
- The magnitude of loss in the second image is much higher, which may indicate a problem with the model or the data.

6 Potential Causes for the Second Image's Spike

- Overfitting to the training data.
- Learning rate might be too high, causing the model to miss the optimal point during optimization.
- A possible bug or anomaly in the data that is only present in the test set or in the particular batch of the test data.
- Insufficient regularization, which might allow the model to fit the noise in the training data too closely.

7 Discussion

We observe from the plots that the modified CNN, while more complex, does not necessarily achieve better performance compared to the baseline. The spike in testing loss for the modified CNN suggests overfitting. The training process and architecture may be further optimized to improve the generalization ability of the model.

8 Conclusion

This analysis demonstrates that increasing the complexity of a CNN does not always lead to better performance and may lead to overfitting. Future work will involve tuning hyperparameters, implementing additional regularization